

Contents

1 R Packages (libraries)	2
1.1 Installation	2
1.2 Loading	2
1.3 The nycflights13 package	3
2 The dplyr package	3
2.1 Selecting features (select())	3
2.2 Filtering observations (filter())	4
2.2.1 The %in% operator	4
2.3 Sorting observations (arrange())	5
2.4 The pipe operator (%>%)	5

1 R Packages (libraries)

Packages in R can extend the capabilities of R, or provide datasets for us to test models against. They extend R's functionality by writing custom functions. Again, we will not focus on writing functions in R, but only on using them well.

1.1 Installation

In this tutorial, we need two packages, `dplyr` (I call it 'deployer') and `nycflights13`. To install them, we can run the `install.packages()` function, with the name of the package we want to install. If the package was already installed, R will attempt to update it.

Alternatively, you may use RStudio's interface, finding the 'Packages' tab or navigating to Tools > Install Packages in the menu bar.

```
install.packages("dplyr")
install.packages("nycflights13")
```

1.2 Loading

Once a package is installed, it must be loaded into R so that it can be used. This is done with the `library()` function.

```
library("dplyr")
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library("nycflights13")
```

Make sure you have loaded these two packages in to R before moving on with the rest of this tutorial.

1.3 The nycflights13 package

The `nycflights13` package provides us with some data, the details of which are mentioned in the tutorial sheet. Apart from that, it does not extend R's functionality. When we loaded it in with `library()`, it should have added a variable `flights` (among others) to the environment. We will look at this dataset for this tutorial.

2 The dplyr package

The `dplyr` package provides helpful functions ('verbs') that will allow us to more easily wrangle (manipulate) our data. In the last tutorial, we did a lot of things with the indexing operation with square brackets. `dplyr` abstracts these indexing operations into several different functions. For this week, we will see:

1. `select()`, to select columns/features of interest
2. `filter()`, to filter out observations based on some criteria
3. `arrange()`, to sort observations based on a column/feature
4. `%>%`, the **pipe operator**, which chains together these dplyr operations.

2.1 Selecting features (`select()`)

Usually in datasets, there are many columns/features that we are not interested in. We can examine the structure of the `flights` dataset with the `str()` function, and see that there are 19 variables.

```
str(flights)
```

```
## tibble [336,776 x 19] (S3: tbl_df/tbl/data.frame)
## $ year      : int [1:336776] 2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
## $ month     : int [1:336776] 1 1 1 1 1 1 1 1 1 1 1 ...
## $ day       : int [1:336776] 1 1 1 1 1 1 1 1 1 1 1 ...
## $ dep_time  : int [1:336776] 517 533 542 544 554 554 555 557 557 558 ...
## $ sched_dep_time: int [1:336776] 515 529 540 545 600 558 600 600 600 600 ...
## $ dep_delay : num [1:336776] 2 4 2 -1 -6 -4 -5 -3 -3 -2 ...
## $ arr_time   : int [1:336776] 830 850 923 1004 812 740 913 709 838 753 ..
## $ sched_arr_time: int [1:336776] 819 830 850 1022 837 728 854 723 846 745 ..
## $ arr_delay  : num [1:336776] 11 20 33 -18 -25 12 19 -14 -
8 8 ...
## $ carrier    : chr [1:336776] "UA" "UA" "AA" "B6" ...
## $ flight     : int [1:336776] 1545 1714 1141 725 461 1696 507 5708 79 301
## $ tailnum    : chr [1:336776] "N14228" "N24211" "N619AA" "N804JB" ...
## $ origin     : chr [1:336776] "EWR" "LGA" "JFK" "JFK" ...
## $ dest       : chr [1:336776] "IAH" "IAH" "MIA" "BQN" ...
```

```
## $ air_time      : num [1:336776] 227 227 160 183 116 150 158 53 140 138 ...
## $ distance      : num [1:336776] 1400 1416 1089 1576 762 ...
## $ hour          : num [1:336776] 5 5 5 5 6 5 6 6 6 6 ...
## $ minute        : num [1:336776] 15 29 40 45 0 58 0 0 0 0 ...
## $ time_hour     : POSIXct[1:336776], format: "2013-01-01 05:00:00" "2013-01-01 05:00:00" ...
```

Usually, we want to narrow down the features to a smaller subset, and we will do so using `dplyr`'s `select()` function. Again, you can use the `?` operator to see a function's documentation and usage. In general, all `dplyr` verbs (functions) will take the dataset we want it to act on as the first argument.

You may follow the code provided in the tutorial sheet. As an exercise, use the `select()` function to select all columns that contain the letter 'c'. (4 columns)

2.2 Filtering observations (`filter()`)

Filtering observations is what we did in the previous tutorial, when we filtered based on gender, for example. `dplyr` provides simpler syntax for filtering than the indexing done with square brackets `[]`, and the ideas are largely the same.

To use `filter()`, we must specify the criteria that we want to filter by, which evaluates to a TRUE or FALSE result.

For example, we could filter for all flights from JFK to LAX by specifying two conditions:

```
# origin and dest referring to the columns in flights
# using the comma and specifying more than one criteria
# acts as an INTERSECTION of criteria.
# gets all flights from JFK to LAX.
filter(flights, origin=="JFK", dest=="LAX")
filter(flights, origin=="JFK" & dest=="LAX")
# equivalent to flights[flights$origin == "JFK" & flights$dest == "LAX",]

# to get a UNION of criteria, use the | operator.
# gets all flights from JFK to anywhere, or from anywhere to LAX.
filter(flights, origin=="JFK" | dest=="LAX")
# equivalent to flights[flights$origin == "JFK" | flights$dest == "LAX",]
```

2.2.1 The `%in%` operator

If we are interested in several values of a feature, we can use the `%in%` operator. Simply put, it checks if the values to the left of `%in%` are *any* of the values to the right of `%in%`. For example, if we were interested in flights departing from JFK or LGA, we can do it in two equivalent ways.

```
filter(flights, origin == "JFK" | origin == "LGA")
filter(flights, origin %in% c("JFK", "LGA"))
```

2.3 Sorting observations (arrange())

This is similar to using the `order()` function to sort observations. In this case, to reverse the sorting, we just need to wrap the feature we want to sort by in a `desc()` function (desc for *descending*).

For example, we want to order flights by their departure delay to see the worst delayed flights.

```
arrange(flights, desc(dep_delay))
```

```
## # A tibble: 336,776 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>      <dbl>      <int>         <int>
##  1  2013     1     9       641           900       1301       1242           153
##  2  2013     6    15      1432          1935       1137       1607           212
##  3  2013     1    10      1121          1635       1126       1239           181
##  4  2013     9    20      1139          1845       1014       1457           221
##  5  2013     7    22       845          1600       1005       1044           181
##  6  2013     4    10      1100          1900        960       1342           221
##  7  2013     3    17      2321           810        911        135           102
##  8  2013     6    27       959          1900        899       1236           222
##  9  2013     7    22      2257           759        898        121           102
## 10  2013    12     5       756          1700        896       1058           202
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
# equivalent to flights[order(flights$dep_delay, decreasing=T),]
```

2.4 The pipe operator (%>%)

The pipe operator is a functionality built in to `dplyr`. To use it, you must import `dplyr`.

The pipe operator allows for quick passing of the output of one function as the input of another. Since `dplyr` verbs all take the data frame we want to act on as the first argument, we can think of the pipe operator `%>%` passing the output of a function as the first argument of the next function.

This allows for chaining of operations in a very natural language manner using `dplyr`. For example, we are interested in only the longest distance flights flown by AA or UA in January, which requires both a filtering and selection.

```
filter(flights, carrier %in% c("AA", "UA"), month == 1) %>%
  select(day, origin, dest, distance) %>%
  arrange(desc(distance))

# equivalent to
# arrange(
#   select(
#     filter(flights, carrier %in% c("AA", "UA"), month == 1),
#     day, origin, dest, distance),
#   desc(distance))
```

There is a very natural language interpretation to the data process when we use the pipe operator. Take the dataframe `flights`, and filter based on some criteria. Then, select the columns specified, and finally sort in descending order of distance.

We call this entire process of performing several steps for a specific purpose is known as a **data pipeline**.

One thing to note is that using the pipe operator `%>%` discard all intermediate results. Thus, if something is wrong in your final result, you may want to check whether something went wrong in each step of the process.

Several exercises: As an exercise:

1. Get all flights with a departure delay more than 30 minutes departing from JFK airport. (15241 rows)
2. Get all flights departing **or** arriving **between** 12pm and 2pm, **and** operated by either AA **or** UA. (18233 rows). You may want to use `between()` function (use `?`) and the `%in%` operator (detailed briefly in the tutorial sheet).
3. Get flights originating from JFK and departing before 8am, and order it based on airtime. (15789 rows)
4. Get flights that have a departure delay more than 30 minutes with a distance less than 2000. (42187 rows)